

Отчёт

Выполнил студент 4 курса ИММиКН им. И.И.Воровича
Красин Александр Сергеевич

Постановка задачи:

1. Подготовка

Изучить optical flow в OpenCV (cv2.calcOpticalFlowFarneback).

Скачать простой датасет: KITTI tracking video (есть боковые виды машин) или YouTube-видео с движущимся транспортом сбоку. Сохранить ссылки на все найденные датасеты.

Подготовить 2-3 короткие последовательности (30-60 кадров).

2. Реализация

Вычислить optical flow между соседними кадрами.

Сохранить результат как цветовую карту (HSV: hue=угол, value=скорость). Тестировать на всех видео.

3. Визуализация

Создать видео с наложенным optical flow (cv2.addWeighted). Сделать 3-5 скриншотов лучших кадров с движением слева/направо. Добавить простую анимацию векторов потока.

Подготовка:

В качестве датасета для работы было взято видео с платформы YouTube - “*Car passing by in a Highway - Royalty Free Stock Video / (Copyright Free) Download*” с канала *Nabulo Sanchar*. Видео содержит движение автомобилей по дороге, при этом одновременно в кадре машины двигаются лишь в одну сторону. Видео позиционируется автором как copyright-free что позволяет свободно его использовать.

Ссылка на видео: <https://www.youtube.com/watch?v=K6xsEng2PhU>

Для дальнейшей работы видео нужно превратить в последовательность кадров. Для этого я воспользовался онлайн инструментом VideoToFrames. В качестве настроек был взят формат png(для минимизации возможной потери качества) и режим сохранения каждого кадра. Оригинальное видео идёт в частоте 30 FPS, итогом получается пронумерованная последовательность из 607 кадров.

Из этих кадров я составил 3 последовательности для работы программы. Каждая последовательность содержит кадры движения одной машины в одну сторону. Кадры каждой последовательности вынесены в отдельную папку.

seq0	Машина с прицепом. Справа/налево.	Кадры 1-60 (60)
seq1	Машина. Слева/направо.	Кадры 197-246 (50)
seq2	Машина. Справа/налево	Кадры 431-490(60)

Реализация:

Алгоритм написан на языке python с использованием библиотеки cv2 для работы с оптическим потоком. Так же в программе используются библиотека numpy для работы с массивами и математических операций и библиотеки glob, os, sys для работы с файлами. Код программы разделён на 4 условных части (в блокноте jupyter прилагаемом к отчёту эти части выделены в разные блоки): конфигурация, инициализация, основной цикл, завершение.

Конфигурация:

В этой части задаются настройки работы программы. Первый блок настроек задаёт директорию для импорта и экспорта файлов. По умолчанию необходимо задать только директорию импорта кадров, после чего названия для экспорта будут созданы автоматически. При необходимости их можно задать вручную. Также можно задать частоту кадров и включить сохранение числовых данных.

Следующий блок задаёт настройки визуализации векторного поля, включая его плотность, масштаб и минимальный порог длины(модуля) вектора для его отображения. Следом идут настройки смешивания исходного кадра и наложенной цветовой карты. Выбор кадров для сохранения скриншотов(важно уточнить, что указываются номера кадров текущей последовательности начиная с 0, а не их номера из исходного видео), а также можно задать параметры, передаваемые в функцию `cv2.calcOpticalFlowFarneback()`.

Инициализация:

На этом этапе программа подготавливается к выводу данных создавая(если отсутствует) директорию для вывода скриншотов, директорию для файлов с данным(если включено) и создаёт экземпляр объекта *VideoWriter* через который будет создаваться визуализация векторов. Для этого в конструктор передаётся кодек формата .mp4, заданная частота кадров и размеры первого кадра импортированной последовательности. Также на этом этапе создаётся список всех изображений в директории для дальнейшей работы.

Основной цикл:

Цикл проходится по всем кадрам последовательности, начиная со второго кадра. Это связано с тем, что мы сравниваем два рядом стоящих кадра. Поэтому первый кадр мы предварительно заносим как “прошлый кадр” и начинаем цикл.

Алгоритм цикла:

1. Преобразование текущего кадра в градацию серого (cv2.COLOR_BGR2GRAY).
2. Вычисление оптического потока между текущим и прошлым кадром.
3. Разделение компонент потока на dx , dy . Переход от декартовых координат к полярным (magnitude, angle_rad), нормализация magnitude. Преобразование угла из радиан в градусы (получение оттенка Hue)
4. Создание HSV-изображения оптического потока, где направление потока задаёт цвет (параметр hue), скорость задаёт яркость (magnitude_norm), а насыщенность задаётся константой = 255. Преобразование HSV в BGR.
5. Наложение цветовой карты на исходный кадр с учётом заданных пропорций $ALPHA/BETA$.
6. Отрисовка векторов. Двухуровневый цикл проходится по всем точкам цветовой карты с заданным в конфигурации шагом и рисует вектор движения между двумя кадрами. При этом векторы с модулем строго меньшим чем пороговое значение отбрасываются. В коде строго заданы цвета для векторов в зависимости от их направления. Векторы со значениями $dx \in (-1, 1)$ считаются вертикальными и им присваивается зелёный цвет.
7. Если номер текущего кадра указан в списке `SCREENSHOT_FRAMES`, то полученное изображение с наложенной цветовой картой и векторами сохраняется в указанной папке с названием “*frame_N.png*”, где N - номер кадра.
8. Если настройка включена, то сохраняются числовые данные кадра (flow, magnitude, angle_rad)
9. Кадр с наложенной цветовой картой и векторами сохраняется в видео.
10. Текущий кадр становится прошлым и начинается новая итерация.

Завершение:

На этом этапе завершается создание видео и создаётся готовый файл, а также освобождаются ресурсы.

Визуализация:

Результатом работы программы для одной последовательности кадров служит видео формата mp4, где на каждый кадр наложена визуализация цветовой карты и векторного поля и отдельно скриншоты заданных кадров. Я применил программу для каждой из 3 последовательностей. В репозитории на github лежат готовые видео для каждой последовательности и скриншоты, разделённые по папка seq0, seq1, seq2 соответственно.

Вывод:

Написанный алгоритм хорошо определяет движущиеся объекты в кадре. На полученных кадрах в цветовой карте легко прослеживаются силуэты автомобилей, что позволяет использовать полученные данные для алгоритмов отслеживания транспорта.